

Manual de Usuario

Directivas Comunes en ARM

1. **.global**
 - **Función:** Define una etiqueta como global para que sea accesible desde otros archivos de código.
 - **Ejemplo:** `.global main` hace que la etiqueta `main` sea accesible globalmente.
2. **.text**
 - **Función:** Indica el inicio de la sección de código ejecutable.
 - **Ejemplo:** `.text` se utiliza para definir la sección donde comienza el código.
3. **.data**
 - **Función:** Indica el inicio de la sección de datos, donde se definen variables o constantes.
 - **Ejemplo:** `.data` se utiliza para declarar la sección de datos.
4. **.word**
 - **Función:** Reserva espacio en memoria para una palabra (normalmente 32 bits).
 - **Ejemplo:** `.word 0x12345678` almacena el valor hexadecimal en memoria.
5. **.align**
 - **Función:** Alinea la siguiente instrucción o dato a una dirección específica en memoria.
 - **Ejemplo:** `.align 4` alinea los datos a un límite de 4 bytes.

Instrucciones comunes en ARM

1. **MOV (Move)**
 - **Función:** Copia un valor en un registro.
 - **Formato:** `MOV destino, fuente`
 - **Ejemplo:** `MOV R0, #5` mueve el valor inmediato 5 al registro R0.
2. **LDR (Load Register)**
 - **Función:** Carga un valor de memoria en un registro.
 - **Formato:** `LDR destino, [fuente]`
 - **Ejemplo:** `LDR R0, [R1]` carga el valor en la dirección de memoria apuntada por R1 en R0.
3. **STR (Store Register)**
 - **Función:** Almacena el valor de un registro en una dirección de memoria.
 - **Formato:** `STR fuente, [destino]`

- **Ejemplo:** `STR R0, [R1]` almacena el valor de R0 en la dirección de memoria apuntada por R1.
4. **ADD (Add)**
- **Función:** Suma dos valores y almacena el resultado en un registro.
 - **Formato:** `ADD destino, operando1, operando2`
 - **Ejemplo:** `ADD R0, R1, R2` suma los valores de R1 y R2 y almacena el resultado en R0.
5. **SUB (Subtract)**
- **Función:** Resta un valor del otro y almacena el resultado en un registro.
 - **Formato:** `SUB destino, operando1, operando2`
 - **Ejemplo:** `SUB R0, R1, R2` resta el valor de R2 del valor de R1 y almacena el resultado en R0.
6. **CMP (Compare)**
- **Función:** Compara dos valores restándolos para establecer las banderas de condición.
 - **Formato:** `CMP operando1, operando2`
 - **Ejemplo:** `CMP R0, R1` compara los valores en R0 y R1 y actualiza las banderas.
7. **B (Branch)**
- **Función:** Salta a una dirección especificada incondicionalmente.
 - **Formato:** `B etiqueta`
 - **Ejemplo:** `B bucle` salta a la etiqueta `bucle`.
8. **BL (Branch with Link)**
- **Función:** Salta a una dirección especificada y guarda la dirección de retorno en el registro de enlace (LR).
 - **Formato:** `BL etiqueta`
 - **Ejemplo:** `BL subrutina` llama a la subrutina y guarda la dirección de retorno.
9. **BX (Branch and Exchange)**
- **Función:** Cambia el flujo de ejecución al valor en el registro especificado y cambia al modo Thumb si es necesario.
 - **Formato:** `BX registro`
 - **Ejemplo:** `BX LR` cambia el flujo de control al valor almacenado en el registro de enlace.
10. **ADR (Load Address)**
- **Función:** Carga una dirección en un registro.
 - **Formato:** `ADR destino, etiqueta`

- **Ejemplo:** `ADR R0, etiqueta` carga la dirección de `etiqueta` en R0.

11. NOP (No Operation)

- **Función:** Instrucción que no realiza ninguna operación, se utiliza para retrasos o alineación.
- **Formato:** `NOP`

12. BR (Branch to Register)

- **Función:** Cambia el flujo de ejecución al valor en un registro.
- **Formato:** `BR registro`
- **Ejemplo:** `BR X0` cambia el flujo de control a la dirección almacenada en el registro X0.

13. SVC (Supervisor Call)

- **Función:** Genera una llamada al sistema.
- **Formato:** `SVC número`
- **Ejemplo:** `SVC 0` invoca una llamada al sistema con el código 0.

14. STP (Store Pair)

- **Función:** Almacena dos registros consecutivos en memoria.
- **Formato:** `STP reg1, reg2, [dirección]`
- **Ejemplo:** `STP X0, X1, [SP, #16]` almacena los registros X0 y X1 en la pila con un desplazamiento.

15. LDP (Load Pair)

- **Función:** Carga dos registros consecutivos desde la memoria.
- **Formato:** `LDP reg1, reg2, [dirección]`
- **Ejemplo:** `LDP X0, X1, [SP, #16]` carga los registros X0 y X1 desde la pila con un desplazamiento.

16. RET (Return)

- **Función:** Retorna desde una subrutina.
- **Formato:** `RET`
- **Ejemplo:** `RET` retorna al llamador de la subrutina.

17. CBZ (Compare and Branch on Zero)

- **Función:** Compara un registro con cero y salta si es igual a cero.
- **Formato:** `CBZ registro, etiqueta`
- **Ejemplo:** `CBZ R0, salida` salta a `salida` si R0 es cero.

18. CBNZ (Compare and Branch on Non-Zero)

- **Función:** Compara un registro con cero y salta si no es igual a cero.
- **Formato:** `CBNZ registro, etiqueta`
- **Ejemplo:** `CBNZ R0, bucle` salta a `bucle` si R0 no es cero.

19. TBNZ (Test Bit and Branch if Not Zero)

- **Función:** Prueba un bit específico de un registro y salta si no es cero.
- **Formato:** `TBNZ registro, bit, etiqueta`
- **Ejemplo:** `TBNZ R0, #1, salida` salta a `salida` si el bit 1 de R0 no es cero.

20. TBZ (Test Bit and Branch if Zero)

- **Función:** Prueba un bit específico de un registro y salta si es cero.
- **Formato:** `TBZ registro, bit, etiqueta`
- **Ejemplo:** `TBZ R0, #1, salida` salta a `salida` si el bit 1 de R0 es cero.

21. AND (Logical AND)

- **Función:** Realiza una operación lógica AND entre dos registros.
- **Formato:** `AND destino, operando1, operando2`

22. ORR (Logical OR)

- **Función:** Realiza una operación lógica OR entre dos registros.
- **Formato:** `ORR destino, operando1, operando2`

23. EOR (Exclusive OR)

- **Función:** Realiza una operación lógica XOR entre dos registros.
- **Formato:** `EOR destino, operando1, operando2`

24. LSL (Logical Shift Left)

- **Función:** Desplaza lógicamente a la izquierda un registro.

25. LSR (Logical Shift Right)

- **Función:** Desplaza lógicamente a la derecha un registro.

26. ASR (Arithmetic Shift Right)

- **Función:** Desplaza aritméticamente a la derecha.

27. ROR (Rotate Right)

- **Función:** Rota los bits hacia la derecha.

28. ADRP (Load Address of Page)

- **Función:** Carga la dirección de una página en memoria.

29. LDUR (Load Register Unprivileged)

- **Función:** Carga un valor de memoria sin privilegios.

30. STUR (Store Register Unprivileged)

- **Función:** Almacena un registro en memoria sin privilegios.

31. MUL (Multiply)

- **Función:** Multiplica dos registros.

32. DIV (Divide)

- **Función:** Divide dos registros.

33. MVN (Move Not)

- **Función:** Copia el valor negado.

34. TST (Test)

- **Función:** Prueba un registro con AND.
- 35. **CMN (Compare Negative)**
 - **Función:** Compara negando.
- 36. **LDRB (Load Register Byte)**
 - **Función:** Carga un byte.
- 37. **UXTB (Unsigned Extend Byte)**
 - **Función:** Extiende un byte a palabra completa.
- 38. **STRB (Store Register Byte)**
 - **Función:** Almacena un byte.
- 39. **SDIV (Signed Divide)**
 - **Función:** División con signo.
- 40. **BLT (Branch if Less Than)**
 - **Función:** Salta si es menor.
- 41. **BEQ (Branch if Equal)**
 - **Función:** Salta si es igual.
- 42. **BLE (Branch if Less Than or Equal)**
 - **Función:** Salta si es menor o igual.
- 43. **BNE (Branch if Not Equal)**
 - **Función:** Salta si no es igual.
- 44. **BGT (Branch if Greater Than)**
 - **Función:** Salta si es mayor.